PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ
FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 2

8va práctica (tipo b)
Primer Semestre 2026

Indicaciones Generales:

- Duración: 110 minutos.

NO SE PERMITE EL USO DE APUNTES DE CLASE, FOTOCOPIAS NI MATERIAL IMPRESO

- No se pueden emplear **variables globales**.
- Tampoco se podrán emplear las funciones malloc, realloc, memset, strtok o strdup, igualmente no se puede emplear cualquier función contenida en las bibliotecas stdio.h, cstdio o similares y que puedan estar también definidas en otras bibliotecas.
- Deberá modular correctamente el proyecto en archivos independientes. LAS SOLUCIONES DEBERÁN DESARROLLARSE BAJO UN ESTRICTO DISEÑO DESCENDENTE. **Cada función NO debe sobrepasar las 20 líneas de código aproximadamente.** El archivo main.cpp solo podrá contener la función main de cada proyecto y el código contenido en él solo podrá estar conformado por tareas implementadas como funciones. En el archivo main.cpp deberá colocar un comentario en el que coloque claramente su nombre y código, **de no hacerlo se le descontará 0.5 puntos en la nota final.**
- El código comentado NO SE CALIFICARÁ. De igual manera NO SE CALIFICARÁ el código de una función si esta función no es llamada en ninguna parte del proyecto o su llamado está comentado.
- Los programas que presenten errores de sintaxis o de concepto se calificarán en base al 40% de puntaje de la pregunta. Los que no muestres resultados o que estos no sean coherentes en base al 60%.
- Se tomará en cuenta en la calificación el uso de comentarios relevantes.
- NO SE PODRÁ EMPLEAR LA CLÁUSULA protected, DE HACERLO NO SE LE CALIFICARÁN LAS CLASES INVOLUCRADAS.

SE LES RECUERDA QUE, DE ACUERDO AL REGLAMENTO DISCIPLINARIO DE NUESTRA INSTITUCIÓN, CONSTITUYE UNA FALTA GRAVE COPIAR DEL TRABAJO REALIZADO POR OTRA PERSONA O COMETER PLAGIO.

NO SE HARÁN EXCEPCIONES ANTE CUALQUIER TRASGRESIÓN DE LAS INDICACIONES DADAS EN LA PRUEBA

- **Puntaje total**: 20 puntos.

INDICACIONES INICIALES

Cree un proyecto de C++ en CLion siguiendo estrictamente las indicaciones que a continuación se detallan:

- La unidad de trabajo será **t:** (Si lo coloca en otra unidad, no se calificará su laboratorio y se le asignará como nota cero)
- Cree allí una carpeta con el nombre **"CO_PA_PN_Lab08_2026_1"** donde **CO** indica: Código del alumno, **PA** indica: Primer Apellido del alumno y **PN** primer nombre (de no colocar este requerimiento se le descontará 3 puntos de la nota final). **Allí colocará el proyecto solicitado en la prueba.**

Cuestionario:

La finalidad principal de este laboratorio es la de reforzar los conceptos contenidos en el capítulo 3 del curso: "Programación Orientada a Objetos". En este laboratorio se trabajará con ESTRUCTURAS AUTORREFERENCIADAS CON OBJETOS.

Deberá elaborar un proyecto denominado "AtencionesPorPrioridadPROG2" y en él desarrollará el programa que dé solución al problema planteado. DE NO COLOCAR ESTE REQUERIMIENTO SE LE DESCONTARÁ 3 PUNTOS DE LA NOTA FINAL. **NO SE HARÁN EXCEPCIONES**

NO PODRÁ EMPLEAR ARREGLOS DE MÁS DE UNA DIMENSIÓN

NO PUEDE MANIPULAR UN PUNTERO CON MÁS DE UN ÍNDICE

LOS ARCHIVOS SOLO SE PUEDEN LEER UNA VEZ

En este laboratorio se continuará trabajando sobre el sistema de gestión de atenciones médicas desarrollado en el **Laboratorio 7**. Para ello, Ud. Cuenta junto con este enunciado del laboratorio con un archivo que contiene la solución del laboratorio anterior.

El objetivo de este laboratorio es incorporar el uso de **estructuras autorreferenciadas** mediante la implementación de una lista doblemente enlazada, que permita organizar las atenciones médicas registradas según una llave numérica.

Para ello, se deberán implementar las siguientes clases: **Par, NodoListaDoble y ListaDoble**

Cada instancia de la clase **Par** representará una asociación entre una llave y un valor.

La **llave** será un número entero construido a partir de la combinación entre la prioridad de la atención y el código de la atención médica.

Por ejemplo, si una atención tiene prioridad **1** y código **5202**, la llave generada será: 15202

Si una atención tiene prioridad **2** y código **3562**, la llave generada será: 23562

El **valor** asociado a cada llave será un puntero a un objeto de tipo **Atencion**, permitiendo mantener el comportamiento polimórfico de las clases derivadas: Atencion *value;

De esta manera, la lista doblemente enlazada deberá permitir almacenar objetos de tipo **Emergencia**, **Urgencia** y **NoUrgencia** utilizando punteros a la clase base **Atencion**.

PARTE 1 (4 puntos)

Implemente la clase **Par**, la cual representa la información almacenada en cada nodo de la lista doblemente enlazada. Esta clase deberá contener:

1. **llave**: valor entero utilizado como criterio de ordenamiento de las atenciones.
2. **valor**: puntero a un objeto de tipo **Atencion**, asociado a la llave correspondiente.

Implemente la clase **NodoListaDoble**, la cual representa un nodo de una estructura doblemente enlazada. Esta clase debe contener los siguientes atributos:

1. **dato**: objeto de tipo **Par** que almacena la información del nodo.
2. **anterior**: puntero autorreferenciado al nodo precedente dentro de la lista.
3. **siguiente**: puntero autorreferenciado al nodo sucesor dentro de la lista.

Implemente la clase **ListaDoble**, la cual representa una lista doblemente enlazada de atenciones. Esta clase debe contener el siguiente atributo:

1. **cabeza** (NodoListaDoble *): puntero al primer nodo de la lista.

Para cada una de las clases implementadas (**Par**, **NodoListaDoble** y **ListaDoble**), deberá definir los constructores, getters, setters y destructores que considere necesarios para garantizar el correcto funcionamiento y gestión de la memoria dinámica utilizada.

Finalmente, agregue a la clase **GestorAtenciones** el siguiente atributo:

1. **atenciones_prioridad** (ListaDoble): lista doblemente enlazada que almacenará las atenciones ordenadas según su prioridad y código.

PARTE 2 (6 puntos)

En esta sección se deberán implementar los métodos necesarios para administrar la estructura **ListaDoble**:

insertar: método que recibe una llave y un puntero a una atención. Este método debe insertar un nuevo **NodoListaDoble** dentro de la lista **atenciones_prioridad**, manteniendo el orden ascendente según la llave. La inserción deberá realizarse mediante un método recursivo auxiliar.

imprimir: método que permite recorrer la **atenciones_prioridad** de manera recursiva e imprimir cada atención registrada, mostrando tanto su llave como la información asociada al valor almacenado.

destruir: método que permite liberar correctamente la memoria dinámica utilizada por la **ListaDoble atenciones_prioridad**, asegurando que la eliminación de sus nodos se realice de manera eficiente y segura.

PARTE 3 (4 puntos)

En esta sección se deberán actualizar la clase base **Atencion** y sus clases derivadas para que sean compatibles con la nueva estructura **ListaDoble**. Para ello, implemente los siguientes cambios:

- Actualice los métodos de lectura de la clase base y de las clases derivadas de ser necesarios.
- Actualice los métodos de impresión de la clase base y de las clases derivadas de ser necesarios.
- Implemente el método virtual puro **conseguir_llave**, encargado de generar la llave de ordenamiento de cada atención según su prioridad y código. Vea ejemplo en la descripción del problema.
- Implemente el método virtual puro **clonar**, encargado de clonar el puntero a atención, dependiendo de cada clase derivada y devolverlo como resultado.
- Actualice los destructores de las clases derivadas, de tal manera que impriman en consola un mensaje indicando que se está ejecutando el destructor correspondiente. Por ejemplo:

```
cout << "Destruyendo <TipoAtencion>" << endl;
```

La llave deberá permitir ordenar las atenciones dentro de la **ListaDoble**.

PARTE 4 (6 puntos)

En esta sección se deberán implementar los métodos de la clase **GestorAtenciones** para el manejo de la lista doblemente enlazada de atenciones priorizadas:

cargar_atenciones_prioridad: método que recibe la ruta de un archivo de atenciones, procesa su contenido y carga las atenciones dentro de la **ListaDoble**, generando para cada atención una llave de ordenamiento según su prioridad y código.

imprimir_atenciones_prioridad: método que recibe la ruta de un archivo de reporte y genera la simulación de atención para la fecha especificada. Las atenciones deberán ser procesadas en el orden definido por la **ListaDoble**, respetando la prioridad de cada atención.

El reporte generado para cada fecha debe presentar un formato similar al siguiente ejemplo:

```
                                        REPORTE DE ATENCIONES PRIORIZADAS
                                          FECHA: 23-01-1990   HORA: 10AM
=============================================================================================================================================
=
EMERGENCIAS
 PRIORIDAD    CODIGO           PACIENTE                     DESCRIPCION      RCP?   NO PULSO          OBSERVACION
     13067      3067     Claudia Benites                Shock Anafilactico      NO         0      Nivel: CRITICO (0 min)
     13070      3070       Berna Montoya       Nacimiento Erasmo G. Montoya      NO         0      Nivel: CRITICO (0 min)
=============================================================================================================================================
=
URGENCIAS
 PRIORIDAD    CODIGO           PACIENTE                     DESCRIPCION        ZONA LESION       RX?             OBSERVACION
     23064      3064       Gonzalo Poma                  Esguince Tobillo       Espalda Baja       NO      Tiempo max: 30 min
     23065      3065       Teresa Molina                   Fractura Brazo             Cabeza       NO      Tiempo max: 30 min
     23066      3066        Pedro Huaman               Torcedura de Muneca               Mano       SI      Tiempo max: 30 min
     23068      3068        Ricardo Leon                  Esguince Tobillo       Espalda Baja       SI      Tiempo max: 30 min
=============================================================================================================================================
=
NO URGENCIAS
 PRIORIDAD    CODIGO           PACIENTE                     DESCRIPCION       ESPECIALIDAD        PROGRAMADA?
     33069      3069      Victor Caceres                Control de Presion       Oftalmologia            NO
```

El reporte en consola para la eliminación puede lucir así:

```
Destruyendo ListaDoble
Destruyendo NoUrgencia
Destruyendo Urgencia
Destruyendo Urgencia
Destruyendo Urgencia
Destruyendo Urgencia
Destruyendo Emergencia
Destruyendo Emergencia
```

Al finalizar la práctica, comprima la carpeta dada en las indicaciones iniciales empleando el programa Zip que viene por defecto en el Windows, no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares.

Profesores del curso: Miguel Guanira, Rony Cueva, Erasmo Gómez, Ana Roncal, Eric Huiza

San Miguel, 19 de Junio del 2026.